TermsGuard AI

Reads the small print and flags the parts that are unfair to you, in plain English.

Cosine similarity alone marks “we do not sell your data” as a violation, so the classifier proposes and a model rules. 102 hand-labelled reference clauses, a 4,527-chunk retrieval index, and up to 15 flagged clauses sent to Claude for verification per analysis. The retrieval index is data-governance and GDPR context rather than regulation, so the tool shows you the passage a flag was drawn from, not an article it claims to quote.

What this evidences: critiquing model quality and evaluation design. The interesting problem is not detection, it is that a purely semantic classifier gets negation exactly backwards, and what you do about it.

Taiwo Alabi · MSc Artificial Intelligence, National College of Ireland

Live demo: https://newtazer-terms-guard-ai.hf.space Source: https://github.com/iamtamilore/terms-guard-ai

---

Contents

1. The problem
2. Results
3. How it works
4. What it compares against
5. Why similarity alone is not enough
6. The verification layer
7. Saying how sure it is
8. Honest limitations
9. Run it
10. Terms used in this report

---

1. The problem

Reading every privacy policy an average internet user meets would take roughly 244 hours a year. Nobody does it. So people agree to data selling and tracking without knowing they have.

The problem is not that the documents are secret. They are published, free, and legally binding. They are simply unreadable at the length and density they are written, and the one clause that matters is buried among forty that do not.

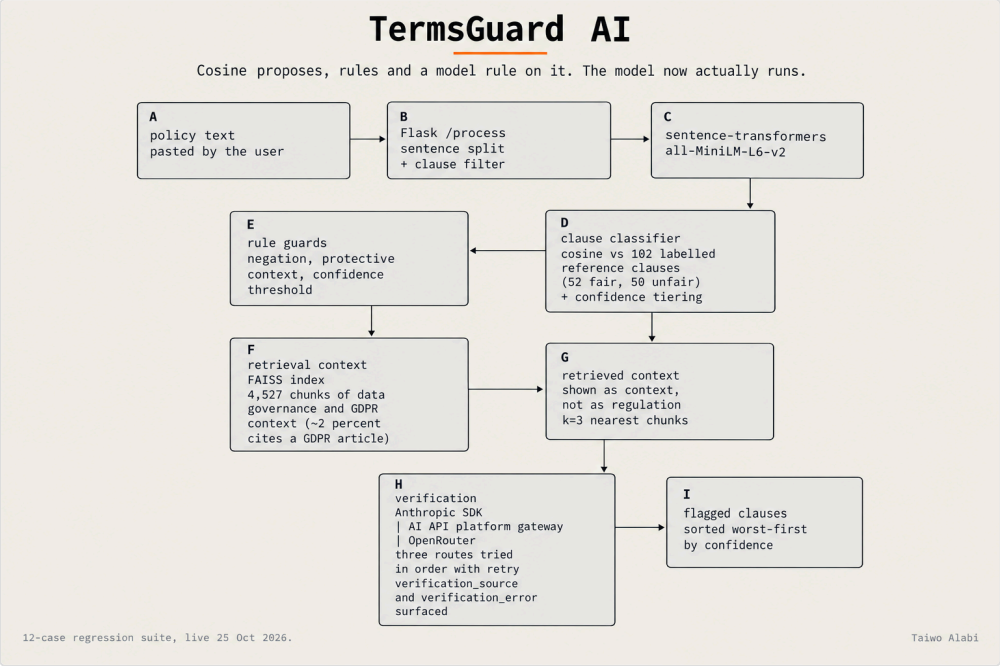
---

2. Results

Measure	Result
End-to-end latency	28 seconds to under 10 for short policies
Reference clauses	102 hand-labelled, 52 fair and 50 unfair
Reference chunks in retrieval index	4,527
Clauses sent to the verification model per analysis	up to 15

The previous build reported that the model had verified each clause, but the verifier had never actually run on the deployed service because the Anthropic key had no credit, the except block swallowed the 400 error, and the app returned cosine scores while still reporting llm_verification_enabled true, so a clause was labelled “Checked by the model” when no model had looked at it. The fix added three routes for the verifier tried in order with retry and a fallback model, and the response now carries verification_source and verification_error so a silent fallback is impossible, verified by a regression suite of twelve inputs passing against the live build with the real model ruling. Two real bugs were only found because the verifier started running: a benign “refund policy” clause was flagged as an Art.82 violation, and a politely worded third-party analytics clause was read as compliant because asking permission was reasoning as lawful; the first is fixed by the prompt stating generic commercial statements are not GDPR clauses, and the second by the prompt stating that politeness does not change what a clause does.

3. How it works



A clause travels this path: similarity proposes, rule guards kill the obvious mistakes, the verifier rules on what is left. The verifier has three routing paths because the previous build silently fell back to cosine and labelled every clause “model-verified” when no model had run. The third path makes that class of bug impossible.

Text is split into individual clauses and de-duplicated. Each clause is turned into a row of numbers by `all-MiniLM-L6-v2`, then compared by cosine similarity, a way of measuring how close two pieces of text are in meaning, against labelled examples of fair and unfair clauses. Anything that looks unfair then has GDPR context pulled for it from a FAISS index (a fast lookup table built for exactly this kind of search), and Claude Haiku double-checks the flag before it is shown.

4. What it compares against

The system carries 102 hand-labelled reference clauses: 52 examples of fair terms and 50 examples of unfair ones. A new clause is judged by how close it sits to each set.

Separately, a FAISS index holds 4,527 chunks of reference material on data governance and GDPR. When a clause is flagged, retrieval pulls the passage closest to it, so the reader can see the context the flag was drawn from rather than taking the model's word.

That split matters. The labelled clauses decide *whether* something is a problem. The GDPR corpus supplies *why*, and it is the second half that makes a flag worth acting on.

---

5. Why similarity alone is not enough

This is the part I would defend hardest in an interview.

Consider two sentences:

> "We sell your personal data to third parties." "We do not sell your personal data to third parties."

To a computer that only measures word patterns, these two sentences look almost identical. They share nearly every word, the same grammar, the same subject. So a system that judges purely by how similar the words look will **flag the second sentence, the reassurance, as a violation**, because it never actually checked which way the sentence points.

So the pipeline carries a layer of explicit rule guards on top of the similarity score:

- Negation detection, for exactly the case above.
- Protective-context detection, for clauses that describe a right being granted rather than removed.
- A children's-data check, because clauses about minors carry obligations that generic similarity scoring does not weight correctly.
- Keyword boosting, for terms whose presence changes the reading regardless of how the sentence embeds.

None of these are clever. They exist because I ran the similarity-only version, read what it got wrong, and found the errors clustered around cases where meaning is inverted by a single word.

---

6. The verification layer

When an API key is configured, Claude Haiku re-reads each flagged clause and removes false positives before anything is shown to the user.

The design decision worth noting is the fallback: with no key set, the tool runs in matching-only mode rather than failing. It degrades to a less precise version of itself instead of refusing to work, which means the demo is always live and the tool is not hostage to a paid dependency.

---

7. Saying how sure it is

Flags are tiered by confidence rather than presented as flat verdicts, with thresholds at 0.55 for high and 0.45 for medium.

A document-level risk assessment then combines the proportion of clauses flagged with the average confidence across them, so a policy with two borderline flags is not reported the same way as one with fifteen strong ones.

This is the same instinct that runs through my other projects: state uncertainty rather than hide it behind a single number.

---

8. Honest limitations

- It is not legal advice. The page says so. It surfaces clauses worth reading properly; it does not tell you your rights.
- The 4,527 chunks in the retrieval index are reference material on data governance and EU GDPR, of which only about two percent cite a GDPR article at all. The retrieved passage is shown as context, not as the article text, because most of the corpus is course material, not the regulation.
- A 12-case regression suite was measured live on 25 October 2026 against the deployed service with the real model ruling. One of the cases ("We offer a refund policy") used to be flagged as an Art.82 violation by the cosine stage. The verifier correctly rejected it after the fix. The remaining cases in the suite pass for reasons the measurements cover: negation handled, French input left uncertain, run-on sentences below the minimum length, and so on.
- Politeness can read as consent. The prompt now states that the tone of a clause does not change what it does.
- This was a contributed project, not a solo build. My work was the retrieval pipeline, the grounding, the rule guards and the latency reduction.

---

9. Run it

Easiest is the live demo: https://newtazer-terms-guard-ai.hf.space

Locally:

```
pip install -r requirements.txt
python server.py        # http://127.0.0.1:7860
```

Optional, for the verification layer:

```
export ANTHROPIC_API_KEY="sk-ant-..."
```

Stack: Python · Flask · sentence-transformers (MiniLM) · FAISS · Anthropic API · Docker

---

10. Terms used in this report

The terms below are defined the way this report uses them, for a reader who wants to check the reasoning rather than take it on trust.

- **Cosine similarity** - a way of measuring how close two pieces of text are in meaning, not just how many words they share.
- **Embedding** - turning a piece of text into a row of numbers, so a computer can compare meaning mathematically instead of just matching words.
- **FAISS index** - a fast lookup table, built for searching millions of embeddings for the closest match almost instantly.
- **Negation** - the word or phrase that flips a sentence's meaning, like turning "we sell your data" into "we do *not* sell your data".